ARMv8-M TrustZone 을 이용한 RTOS 환경에서의 버퍼 단위 DMA 보호 기법

Buffer-Granularity DMA Protection for RTOS Using ARMv8-M TrustZone

요 약

DMA(Direct Memory Access)는 CPU 를 거치지 않고 메모리에 접근하는 독립적인 버스 마스터로, MPU(Memory Protection Unit), DWT(Debug and Watchpoint Trace unit) 등 CPU 의 접근만 검사하는 보호 수단을 우회한다. 이를 막기 위해 DMA 요청을 MPU 영역 단위로 검증하는 기법이 제안되었으나, 이 방식은 DMA 버퍼와 일반 데이터를 구분하지 못해 같은 태스크 데이터의 오염과 전송 중 경계 초과를 막지 못한다. 본 논문은 ARMv8-M TrustZone 의 하드웨어 기능을 이용하여 DMA 버퍼 단위 요청 시점 검증과 DMA 전송 중 경계 초과를 탐지 · 차단하는 전송 시점 보호를 결합한 기법을 제안한다.

1. 서 론

저사양 임베디드 기기에서는 실시간성·저전력 설계가 필수적이기 때문에 CPU 개입 없이 데이터를 옮기는 DMA 가 핵심 수단으로 활용되고 있다[1]. 그러나 DMA 는 CPU 와 독립된 버스 마스터이므로 MPU, DWT 워치포인트 등 CPU 의 접근만 검사하는 보호 수단을 우회한다. 서버나 데스크톱에서는 DMA 를 이용한 공격을 막기 위해 IOMMU(Input-Output Memory Management Unit)와 같은 보호 하드웨어를 사용한다. 그러나 일반적인 MCU 에는 IOMMU 가 없어 DMA 공격에 취약하며, 이러한 공격은 두 가지 경로로 발생할 수 있다. 첫째는 CVE-2025-5688[2]처럼 원격에서 트리거 가능한 메모리 오염 취약점을 이용해 DMA 요청 인자(주소, 길이)를 조작하는 경로이다. 둘째는 전송 중 경계 초과(Overflow)로, CVE-2026-10644[3]처럼 요청 인자가 정상이더라도 드라이버 결함으로 인해 DMA 가 버퍼 경계를 넘어 접근하는 경로이다.

이와 같은 DMA 를 이용한 공격을 막기 위한 선행 연구로 D-Box[1]가 있다. D-Box 는 FreeRTOS-MPU 환경에서 별도의 특권 태스크가 DMA 요청을 검증한다. 하지만 검증 단위가 MPU 영역이므로 같은 태스크 영역의 내부 데이터로 향하는 접근을 막지 못하고, 검증이 요청 시점에 한정되어 전송 중 경계 초과도 다루지 못한다. 이러한 한계를 해결하기 위해서는 DMA 버퍼 단위로 요청을 검증하고 전송 중 경계 초과까지 탐지하는 기법이 필요하다.

버퍼 단위의 실시간 검증은 고사양 시스템에서 먼저 제안된 바 있다. DMAAuth[4]는 버스에 배치한 전용 인증 하드웨어로 모든 DMA 접근을 버퍼의 바이트 단위 경계와 비교하여, 등록된 버퍼 밖으로의 접근을 전송 중에도 효과적으로 차단한다. 하지만 이는 상용 MCU 에 적용할 수 없는 전용 하드웨어를 전제로 한다.

이에 본 논문은 상용 ARMv8-M MCU 의 TrustZone 하드웨어를 이용한 DMA 보호 기법을 제안한다. 이는 Secure DMA 중재자와 Secure 가드 블록으로 구성된다. 중재자는 요청 시점에 인자를 버퍼 단위로 검증하고, 가드 블록은 전송 중 발생하는 경계 초과를 탐지 · 차단함으로써, 메모리 보호 기법을 사용하는 MCU 환경에서의 DMA 보호를 구현한다.

제안하는 보안 기법에서는 Secure 펌웨어, Non-secure 커널, 부트 과정을 신뢰 기반(TCB, Trusted Computing Base)으로 두고, 공격 모델을 다음과 같이 가정한다. 첫째, 공격자는 Non-secure 태스크의 메모리 오염 취약점을 이용하여 DMA 요청 함수에 전달되는 주소와 길이를 임의로 조작할 수 있다. 둘째, 공격자는 주변장치 드라이버의 결함 등을 이용하여 전송 중 경계 초과를 유발할 수 있다. 이러한 공격 모델에 대응하여 본 기법의 보호 목표는 요청 태스크에 등록된 DMA 버퍼 밖, 즉 커널 데이터, 다른 태스크의 메모리, 같은 태스크의 DMA 버퍼 외 데이터로 향하는 DMA 접근을 탐지하고 차단하는 것이다. 이때, 물리 공격과 부채널 공격은 이 방어 범위에서 제외한다.

본 논문의 이후 구성은 다음과 같다. 2 절에서는 제안 기법에 필요한 ARMv8-M TrustZone 의 기능들과 관련 연구의 한계점을 설명하고 3 절에서는 제안하는 설계에 대해서 설명한다. 4 절에서는 구현 및 평가에 대해 설명한다. 5 절에서는 결론을 서술한다.

2. 관련 배경 및 연구

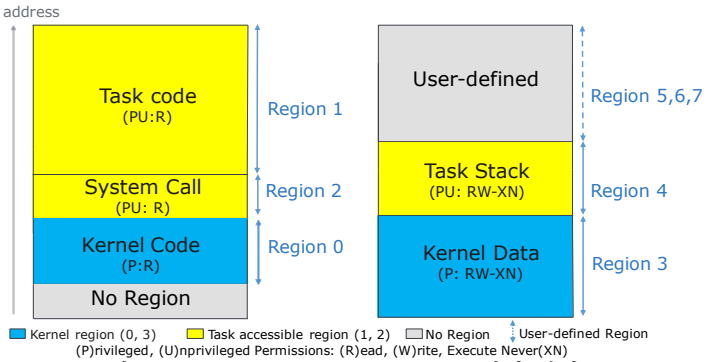
본 절에서는 DMA 위협과 제안 기법을 구현하는데 필요한 하드웨어 보호 기능을 소개한다. 기준 아키텍처로는 최근 상용 MCU 에 널리 탑재되는 TrustZone 을 지원하는 ARMv8-M 을 선택하였다. 이어서 실행 환경을 보안 영역과 비보안 영역으로 나누는 기능, 메모리 영역별 접근 권한을 지정하는 기능, 버스 수준에서 메모리 접근을 제어하는 기능을 차례로 설명한다.

2.1 DMA 의 요청 시점과 전송 시점

요청 시점에는 DMA 드라이버 API 등을 이용하여 채널에 목적지 주소, 전송 길이를 설정하고 채널을 활성화한다. 전송 시점에는 DMA 컨트롤러가 CPU 개입 없이 주소를 증가시키며 데이터를 옮긴다.

2.2 ARMv8-M TrustZone 과 TT 명령어

ARMv8-M 에서의 TrustZone 은 메모리와 주변장치를 Secure 와 Non-secure(NS) 영역으로 구분하며, NS 코드는 NSC(Non-secure Callable) 영역에 정의된 진입점을 통해서만 Secure 함수를 호출할 수 있다. MPU 는 보안 상태에 따라 Secure MPU(S-MPU)와 NS MPU(NS-MPU)로 분리되어 독립적으로 동작한다[5]. 예를 들어, NS 에서 동작하는 코드는 NS-MPU 설정에 의해 메모리에 접근 권한이 달라지며, S-MPU 는 관여하지 않는다. 이와 같은 특징을



[그림 1] FreeRTOS-MPU(PMSAv8) MPU 영역 설정

이용하여 추가적인 테이블을 관리하지 않고 DMA 버퍼를 하드웨어적으로 기록할 수 있다.

ARMv8-M 에서 제공하는 TT 계열 명령은 다음과 같다. TT(Test Target)은 주소 값을 인자로 받아서, 현재 보안 상태(Secure, Non-Secure) 및 권한(Privileged, Unprivileged)에 해당하는 MPU 속성을 조회하며, TTT(TT Unprivileged)명령은 Unprivileged 권한으로 현재 보안 상태에 해당하는 MPU 속성을 조회한다. 반면, Secure 상태에서 실행되는 TTA(TT Alternate domain)는 주소 값에 대해 NS-MPU 의 속성을 조회하며, TTAT(TTA Unprivileged)는 Unprivileged 권한으로 NS-MPU 속성을 조회 할 수 있다.

본 논문에서 사용한 STM32L562E-DK 의 GTZC(Global TrustZone Controller)는 TZSC(TrustZone Security Controller)로 주변장치(DMA 채널 포함)를, MPCBB(Block-Based Memory Protection Controller)로 SRAM 을 256 바이트 블록 단위로 Secure 또는 NS 로 지정한다. 또한, TrustZone 에서는 하드웨어에서 발생하는 모든 트랜잭션에 대해서 보안 상태에 대한 정보를 추가할 수 있다. 이때 CPU 에서 발생하는 트랜잭션 뿐만 아니라, DMA 와 같이 독립적인 버스 마스터에서 발생하는 트랜잭션 또한 보안 상태 설정이 가능하다. NS 상태로 설정된 트랜잭션은 Secure 자원에 접근할 수 없고, TZIC(TrustZone Illegal access Controller) 인터럽트를 발생시킨다[6]. 이 동작은 MPCBB 로 설정된 Secure 블록에 접근할 때도 동일하게 발생한다. 이러한 특징을 이용하여 DMA 전송 중 탐지를 구현할 수 있다.

2.3 FreeRTOS-MPU

제안 기법은 FreeRTOS 환경에서의 MPU 영역 설정을 전제로 한다. FreeRTOS-MPU 는 [그림 1]과 같이 NS-MPU 의 8 개 영역 중 5 개를 커널 코드·데이터, 시스템 콜, 태스크 코드·스택에 고정으로 사용하고, 태스크마다 3 개의 사용자 영역만 남긴다. 이때, 스택·사용자 영역은 Context Switching 시 태스크별로 재설정된다.

2.4 관련 연구

메모리 격리 환경에서의 DMA 보안 연구는 여러 방향으로 진행되어 왔다. 임베디드 기기를 대상으로 하는 D-Box[1]에서는 특권 DMA 태스크가 비특권 태스크의 DMA 요청을 MPU 영역과 주변장치 권한을 기준으로 검증한다. 그러나 검증 단위가 MPU 영역이어서 같은 태스크 영역 내부 데이터의 오염은 막지 못하고, 전송 중 경계 초과도 다루지 않는다.

이처럼 검증 단위가 보호 대상보다 커서 생기는 문제는 MCU 뿐 아니라 고사양 시스템에서도 나타난다. 서버나 데스크톱의 경우 IOMMU 를 사용하여 DMA 공격 보호를 제공하는데, 이는 DMA 접근을 페이지 단위로만 제한하여 같은 페이지의 다른 데이터까지 노출한다. DMAAuth[4]는 이를 해결하기 위해 버퍼의 바이트 단위 경계로 DMA 포인터에 서명하고, I/O 버스의 전용 하드웨어인 Authenticator 가 모든 DMA 접근을 검사하여 버퍼 단위 보호를 제공한다. 그러나 이 기법은 전용 하드웨어가 필요하여 상용 MCU 에는 적용할 수 없다.

기법 \ 대응 범위	타태스크·커널	영역 내부 보호 객체	전송 중 경계 초과	상용 MCU 적용
MPU+DWT[7]	X	X	X	O
D-Box[1]+DWT[7]	O	X	X	O
DMAAuth[4]	O	O	O	X
제안 기법	O	O	O	O

[표 1] DMA 위협 유형별 대응 범위 비교

MCU 에서는 전용 하드웨어가 없을 뿐만 아니라 MPU 영역 수도 제한된다. 따라서 MPU 영역 내부에서 DWT 와 같은 메모리 감시 수단을 사용하여 특정 객체에 대한 세부 권한을 지정하기도 한다[7]. 그러나 STM32L562E-DK 에서 확인한 결과 DWT 또한 MPU 와 마찬가지로 DMA 쓰기에 반응하지 않았다. 이와 같이 DWT 보호 객체가 쓰기가 허용된 MPU 영역에 별개로 존재한다면, DMA 는 MPU 영역 검증을 통과한 상태로 내부 보호 객체를 변조할 수 있다. 이처럼 영역 내부에서 변조가 가능하므로, MPU 영역 단위가 아닌 더 세밀한 DMA 버퍼 단위의 검증이 필요하다.

위와 같은 DMA 보호를 위한 관련 연구들은 [표 1]에 나타나 있듯이, 상용 MCU 에 적용 가능한 기존 기법으로는 MPU 영역 내부의 보호 객체를 향한 DMA 접근과 전송 중 경계 초과를 방어하지 못한다. 따라서 본 논문에서는 상용 MCU 가 제공하는 기능을 사용하여 DMA 접근을 버퍼 단위로 검증하고, 전송 중 경계 초과를 차단할 수 있는 보호 기법을 제안한다.

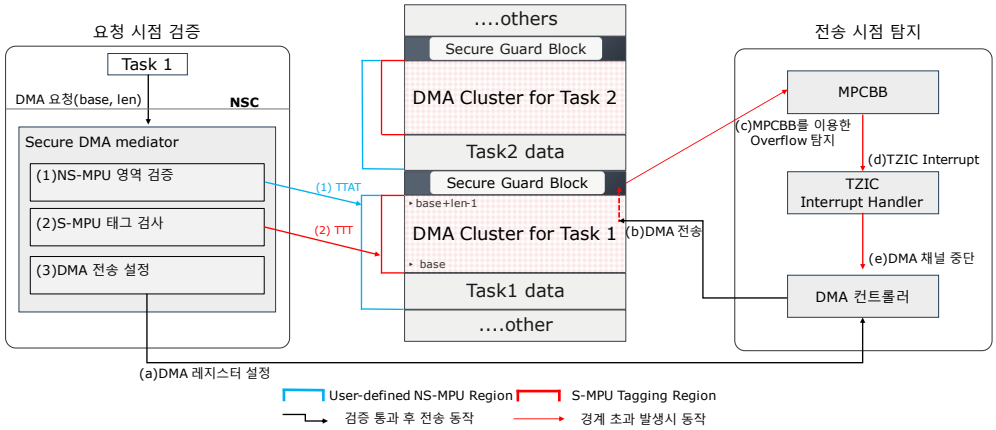
3. 제안 기법

3.1 메모리 배치

제안 기법의 효율적인 적용을 위해서는 [그림 2]와 같이 메모리를 배치하여야 한다. 즉, 태스크마다 DMA 버퍼를 하나의 연속 구간인 DMA Cluster(이하 Cluster)로 모으고 태스크 전역 데이터와 인접하게 배치한다. 이 메모리 배치는 태스크마다 각각 하나의 S-MPU 영역과 MPCBB 가드 블록을 사용하여 제안 기법을 적용할 수 있게 한다. 메모리 배치 이후, 각 태스크의 Cluster 를 별개의 S-MPU 영역으로 설정한다. 이러한 S-MPU 영역 설정은 Secure DMA 중재자가 TTT 명령어로 조회할 수 있는 태그로 사용할 수 있게 한다. 또한, 클러스터 위쪽(DMA 주소 증가 방향)의 MPCBB 가드 블록은 전송 중 경계 초과(Overflow)를 TZIC Interrupt 로 탐지 · 차단한다.

3.2 요청 시점 검증: Secure DMA 중재자

DMA 컨트롤러와 채널을 모두 TZSC 로 Secure 에 할당하여 NS 코드가 DMA 를 직접 설정하지 못하게 하고, DMA 요청 (base, len)에 대해서는 Secure DMA 중재자가 다음을 확인한다. [그림 2]의 (1)에서는, 호출자 권한을 확인한다. 그 후 권한에 맞는 TTAT(비특권) 혹은 TTA(특권) 명령을 이용하여 NS-MPU 에서 양 끝점 base 와 base+len-1 이 호출자의 권한으로 쓰기 가능한지 확인한다. (2)에서는 TTT 명령을 사용하여 양 끝점의 S-MPU 영역 번호를 조회하여 둘 다 유효하고 같은 영역인지 확인한다. S-MPU 영역은 연속 구간이므로 이를 만족하면 전송 범위 전체가 Cluster 안에 있음을



[그림 2] 제안 기법 메모리, 보호 수단 배치 및 검증, 탐지 흐름

검증할 수 있다. 검증을 통과한 DMA 요청에 대해서 DMA 전송이 NS 트랜잭션으로 수행하도록 설정하여 3.3 절의 MPCBB 가드 블록이 적용되게 한다. 버퍼 단위 검증은 등록된 버퍼 수와 무관하게 TTT 명령 2 회로 끝나는 대신, 태그 수는 S-MPU 영역 수(8 개)로 제한된다.

3.3 전송 시점 보호: MPCBB 가드 블록

요청 시점 검증은 인자만 검증할 수 있으므로, DMA 전송 중 발생하는 경계 초과[3]는 탐지할 수 없다. [그림 2]의 (b)-(e)는 MPCBB 가드 블록이 이 오류를 하드웨어 수준에서 탐지하는 과정을 나타낸다. 경계를 넘는 NS DMA 쓰기 및 읽기 시, TZIC 인터럽트 핸들러에서 DMA 채널을 중단한다. MPCBB 가드 블록에는 NS 데이터를 두지 않음으로써 NS 영역의 정상적인 동작이 가능하도록 한다.

4. 구현 및 평가

STM32L562E-DK(Cortex-M33, 110 MHz)에서 NS 측은 FreeRTOS V11.3.1(MPU wrappers v2), Secure 측은 베어메탈로 구현하였다. DMA 버퍼는 매크로를 사용하여 태스크별 섹션에 수동으로 모은다. 링커 스크립트는 태스크 전역 데이터, 클러스터, 가드 블록 순서로 정렬하여 배치하고, 빌드 시점에 DMA 버퍼 정보를 저장한다. 부팅 시 클러스터마다 S-MPU 영역을, 가드 블록마다 MPCBB Secure 속성을 설정하고, TZIC 인터럽트 발생 시 DMA 전송을 중단하도록 설정한다. DMA 채널은 Secure 에 두어 NS 코드가 직접 설정할 수 없게 하고, Secure DMA 중재자에 의해 검증된 요청에 한해서 DMA 전송을 NS 로 설정하여 전송을 수행한다.

버퍼 단위 검증에 대한 사이클 측정은 캐시가 채워진 상태에서 8 바이트 메모리 간(M2M) 전송을 32 회 반복하여 측정하였다. [표 2]는 S-MPU 태그 방식과 Flash 테이블 방식의 검증 비용을 DWT 사이클 카운터로 측정한 결과이다. TTT 2 회로 S-MPU 태그를 확인하는 방식은 8 개 항목의 Flash 테이블을 탐색하는 방식보다 평균 63 사이클(약 28%) 빨랐다.

전송 시점 보호를 확인하기 위해, 검증을 통과한 요청에 대해 DMA 전송 길이를 Cluster 를 넘어서게 변조한 공격을 수행하였다. 그 결과 DMA 가 가드 블록에 처음 접근하는 시점에 TZIC 인터럽트가 발생하여 핸들러가 DMA 채널을 중단하였고, MPCBB 가드 블록을 초과하는 메모리는 변조되지 않았다. 쓰기와 읽기 방향 모두에서 경계 초과가 동일하게 탐지 · 차단되었다.

측정 항목	평균	비고
빈 NSC 진입·복귀	66	상태 전환 약 58
검증 포함 (테이블 탐색)	223	최대 525
검증 포함 (S-MPU 태그)	160	TTT×2, 31/32 회 동일

[표 2] 중재자 경로 비용 (단위: 사이클)

5. 결론 및 향후 연구

본 논문은 상용 MCU 에서 기존 기법이 다루지 못한 두 가지 DMA 위협, 즉 같은 MPU 영역 안의 보호 객체를 향한 DMA 접근과 전송 중 경계 초과를 막는 DMA 보호 기법을 제안하였다. 제안 기법은 요청 시점에는 S-MPU 태깅으로 DMA 인자를 버퍼 단위로 검증하고, 전송 시점에는 MPCBB 가드 블록으로 경계 초과를 버스 수준에서 차단한다. 이 과정에서 NS-MPU 영역을 추가로 사용하지 않으므로, 기존 FreeRTOS-MPU 의 태스크 격리 구성을 그대로 유지할 수 있다. 구현에서 제안 기법은 두 위협 시나리오를 모두 탐지·차단하였으며, 버퍼 단위 검증은 테이블을 이용한 방식보다 빠르고 측정 편차도 거의 없었다.

그러나 제안 기법은 태스크당 가드 블록 256 바이트와 S-MPU 영역 1 개를 추가로 사용하므로, S-MPU 를 이미 다른 용도로 쓰는 환경에서는 Flash 에 저장하는 테이블 방식으로 대체해야 한다. 따라서 향후에는 가드 블록을 Secure 저장소로 재활용하여 메모리 오버헤드를 줄이고, 오픈소스 펌웨어에 적용하여 제안 기법의 실용성을 검증할 계획이다.

참고문헌

[1] A. Mera, Y. H. Chen, R. Sun, E. Kirda, L. Lu, "D-Box: DMA-enabled Compartmentalization for Embedded Applications," Proc. of the Network and Distributed System Security Symposium (NDSS), 2022.
[2] CVE-2025-5688: Out-of-bounds write in FreeRTOS TCP, Security Bulletin AWS-2025-012, 2025.
[3] CVE-2026-10644: Out-of-bounds write in Microchip SERCOM-G1 UART driver, Zephyr Project, 2026.
[4] X. Wang, W. Shen, Y. Bu, J. Zhou, Y. Zhou, "DMAAuth: A Lightweight Pointer Integrity-based Secure Architecture to Defeat DMA Attacks," Proc. of the 33rd USENIX Security Symposium, 2024.
[5] Arm Ltd., "Armv8-M Architecture Reference Manual," DDI 0553.
[6] STMicroelectronics, "RM0438: STM32L552xx and STM32L562xx Arm-based 32-bit MCUs Reference Manual."
[7] X. Tan, S. Mohan, Z. Zhao, “WATSON: Leveraging Data Watchpoints for Shadow Stack Protection on Embedded Systems,” arXiv:2605.08604, 2026.